Assignment No-1

DD MM YYYY

1. Define Data structure? with a neat diagram, explain the classification of Data structure with example?

A:- The study of, how the data can be collected and stored in main memory during execution.

- How the data can be represented
- How the data is organized or how the data is categorized.
- How efficiently the data can be retrieved and manipulated.

	Names	Marks	Grade
A[0]	AMITABH	100	'B'
A[1]	SACHIN	101	'B'
A[2]	ARJUN	102	'A'
A[3]	BHIM	103	'A'
A[4]	MODI	104	'A'

n=5 arranged in ascending order

=> The data structures are classified into:

1) Primitive data structures:-

The data structures that can be manipulated directly by machine instructions are called primitive data structures.

Ex:- * integers (int)
* floating point numbers (float)
* characters (char)
* double values (double)
* pointers.

all are primitive data structures in c language.

2) Non-primitive data structure:-

D D M M Y Y Y Y

The data structures that cannot be manipulated directly by machine instructions are called non-primitive data structure.

Ex: * arrays
* Stacks
* queues
* linked lists
* trees.

2. Explain the memory allocation. Explain Dynamic memory allocation functions supported by 'c' with syntax and example?

A: => Memory is a hardware unit where the data or instructions (programs) are stored. The process or mechanism by which memory space is allocated to variables & constants during run time when the program is being executed is called memory allocation.

=> There are two memory allocation:-

* Static memory allocation:-
allocation of memory during compile time.
Ex:- int x, float y;

* Dynamic memory allocation:-
allocation of memory during runtime

=> There are 4 memory management function in C to support dynamic memory management defined in the header file <stdlib.h>

1) Malloc ()
2) calloc ()
3) realloc ()
4) free ()

=> Dynamic memory allocation functions.

1) Malloc
This
- le block
requir
=> Syntax
Declara
void
Ex:- #inclu
#inclu
int ma
int *
flo
pi =
pf =
* pi
* p
prin
free
free
}

2) Calloc (
Calle
blocks
=> Syntax
=> prototy
void *

=> Functio
ptr =

DD MM YYYY

1) Malloc (Size);-

This function allows the program to allocate sin
-le block of memory of requested size as & when
required & the exact amount needed during execution

=> Syntax.

Declaration:

```
void* malloc (size_t size);
```

Ex:-

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int *pi;
float *pf;
pi = (int*) malloc (sizeof(int));
pf = (float*) malloc (sizeof(float));
*pi = 10.24;
*pf = 3.14;
printf ("an integer = %d, a float = %f\n", *pi, *pf
free (pi);
free (pf);
}
```

2) Calloc (n, size);

Calloc stands for configuous allocation of mult
blocks is mainly used to allocate memory for arr

=> Syntax:

=> prototype Declaration:

```
void* calloc (size_t element count, size_t element-
                size);
```

=> Function call

```
ptr = (data_type*) calloc (n, size)
```

D D M M Y Y Y Y

Ex:- ptr = (int *) calloc (5, sizeof (int))

3. Realloc (ptr, size):-
=> Before using this function, the memory should have been allocated using malloc(), calloc().
=> Sometimes, the allocated memory may not be sufficient as we may requires additional memory space.
=> Syntax:
prototype Declaration:-
void * realloc (void * ptr, size_t newsize);
=> Function call: ptr = (data_type*) realloc (ptr, size)
ptr = (int*) realloc (ptr, 1* sizeof (int));
ptr = (int*) realloc (ptr, 4);

4) Free (ptr):-
This function is used to de-allocate or free the allocated block of memory which is allocated by using the functions calloc(), malloc() & realloc()
=> prototype declaration
void free (void * ptr);
Ex:- #include <stdlib.h>
free (ptr);
ptr = NULL;

3. Explain structure and union with an example? (Differentiate both) List out differentiate structure and union.

DD MM YYYY

A: Structure :-

A Structure is a collection of one or more declaration of variables of same data type or dissimilar data types, grouped together as a single entity.

⇒ Syntax:

```
Struct
{
type 1 member 1;
type 2 member 2;
};
```

Exc:-

```
Struct
{
char name [20];
int marks;
float average;
};
```

⇒ Union :-

A union is a collection of one or more declaration of variables of same data type or dissimilar data types, grouped together as a single entity.

⇒ Syntax

```
union
{
type 1 member 1;
type 2 member 2;
};
```

Exc:-

```
union
{
char name [20];
int marks;
float doubles;
};
```

Differentiating of structure and union.

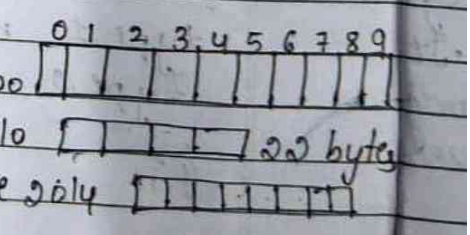
Structure	Union
* Separate memory locations are allocated for every member of the structure	* The memory is allocated equal to its size is equal to maximum size of a member.

DD MM YYYY

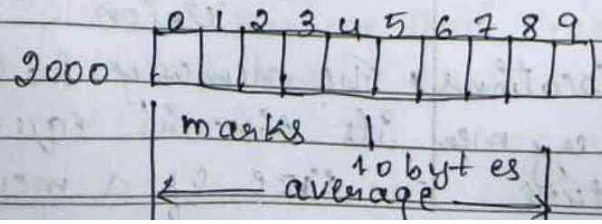
Structure	Union
* Each member within a structure is assigned unique address.	* The address is same for all members.
* The address of each member is greater than the address of its previous member.	* The address is same for all members.
* Altering the value of one member will not affect other members of the structure.	* Altering the value of one member affects other members (affects) as the memory is shared
* Several members of a structure can be initialized.	* Only the first member of the union can be initialized.
* Size of structure is >= sum of sizes of its members	* Size of union is = size of largest member.

=> Structure :- Struct CSE

```
{
char name[10];
int marks;
double average;
} cse;
```



=> Union :-




```
Union
{
char name[10];
int marks;
```

double average;
}case;

4. Explain any four string handling function supported by 'C' with syntax & example.

A:- String handling functions:-

* Strlen(Str) :- Returns the no. of characters in the given string.
* Strcpy(Dest, Src) :- Copies the given source string to the destination string.
* Strcmp(S1, S2) :- compares two strings & returns 0 if two strings are equal 1 if $S_1 > S_2$, -1 if $S_1 < S_2$.
* Strcmp(S1, S2, n) :- Compares n characters from S1 & S2 returns 0 if they are equal 1 if n characters of S2.
* Strupr(Str) :- converts characters of Str into upper case overwrites on Str.
* Strlwr(Str) :- converts characters of Str into lower case overwrites on Str.
* Strrev(Str) :- reverse a given string Str & overwrites on Str.

function	Description.
* char *Strcat(char *dest, char *Src)	concatenate dest & Src string return results in dest.
* char *Strncat(char *dest, char *Src, int n)	concatenate dest & n characters from Src; return result in dest
* Strcmp(char *Str1,	Compare two strings return <0

D D M M Y Y Y Y

char * Str2)	Str1 < Str2 ; 0 if Str1 = Str2; > 0 if Str1 > Str2.
* Strcmp (char * Str1, char * Str2, int n)	compare first n characters return -n < 0 if Str1 < Str2; 0 if Str1 = Str2; > if Str1 > Str2

5. what do you mean by pattern matching? outline the Knuth morrieus pratt (KMP) algorithem & illustrate it to find the occurares of the following pattern.
P: ABCDABD
S: ABC ABCDAB ABCDABC DABDE

A:- pattern matching algorithms:-
It is the problem of deciding wheather or not a given string pattern p appears in a string text T. The length of p does not exceed the length of T.

The Knuth morris pratt (KMP):-
Algorithm is an efficient string matching algorithm used to search for a pattern within a text. It uses a preprocessing step to handle mismatches smartly & achieves linear time complexity. KMP was developed by Donald Knuth.

Technologies used in KMP algorithm:-
i) text (txt):- The main string in which we want to search for a pattern.
ii) pattern (pat):- The substring we are tuying to find within the text.
iii) Match :- match occurs when all characters of the

pattern align exactly with a substring of the text.

iv. LPS Array (longest prefix suffix): for each position i in the pattern LPS[i] stores the length of the longest proper prefix which is also a suffix in the substring

v. Proper prefix :- A proper is a prefix that is not equal to the whole string.

vi. Suffix :- A Suffix is a substring that ends at the current position.

P: ABCDABD

Q: ABC ABCDAB ABCDABC DABDE

Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Text :- ABC ABC DABABCDABCDABDE

LPS Table

i	P[i]	LPS[i]
0	A	0
1	B	0
2	C	0
3	D	0
4	A	1
5	B	2
6	D	0

KMP Scanning find match starting at index 15 - 7 + 1 = 9

* Patter (P) = "ABCDABD"
* Text (Q) = "ABC ABCDAB ABCDABC DABDE"
* Match found at index 9

6. Write a program in C to implement push, pop & display operation in stack using arrays.

A:-
```
#include <stdio.h>
#define SIZE 5
int stack[SIZE];
```

```
int top = -1;
void push (int value) {
if (top == SIZE - 1) {
printf ("Stack overflow! cannot push %d \n", value);
} else {
top ++;
stack [top] = value;
printf ("%d pushed to stack. \n", value);
}
}
void pop () {
if (top == -1) {
printf ("Stack underflow! no elements to pop \n");
} else {
stack [top] );
top--;
}
}
void display () {
if (top == -1) {
printf ("Stack is empty. \n");
} else {
for (int i = top ; i >= 0; i--) {
printf ("%d \n", stack [i]);
}
}
}
int main () {
int choice ; value;
while (1) {
```

7.

DD MM YYYY

```
printf ("\n--- Stack Menu ---\n");
printf ("1. push\n");
printf ("2. pop\n");
printf ("3. Display\n");
printf ("4. Exit\n");
printf ("Enter your choice:");
Scanf ("%d", &choice) {
case 1:
printf ("Enter the value to push:");
Scanf ("%d", &value);
push (value);
break;
Case 2:
 pop();
break;
case 3:
display ();
break;
Case 4:
printf ("Exiting program...\n");
 return 0;
 default;
printf ("Invalid choice please try again\n");
   }
  }
 }
```

7. Write function in C for the following operations out using built in function.

i) Compare two strings.

D D M M Y Y Y Y

ii] Concatenate two strings
iii] Reverse string

i] compare two strings.

```
int compare strings (char str1[], char str[]) {
  int i=0;
while (str1[i] != '\0' && str2[i] != '\0') {
 if (str1[i] != str2[i])
return str1[i] = str2[i];
  i++;
  }
return str1[i] - str2[i];
}
```

ii] Concatenate two strings

```
void concatenate strings (char str1[], char str2[]) {
int i=0, j=0,
while str1[i] != '\0'.
  i++
while (str2[j] != '\0') {
str[i] = str2[j];
 i++;
 j++;
}
 str1[i] = '\0';
}
```

3. Reverse a string

```
Void reverse string (char str[]) {
int i=0, j=0, temp;
while (str[j] != '\0')
 j++;
```

```
j--;
while (i<j) {
temp = Str[i];
Str[i] = Str[j];
Str[j] = temp;
i++;
j--;
}
}
```

8. Explain the representation of linear array in mem y also consider the linear arrays AAA(5:50) & BBB(5:50) & BBB(-5:10).

i. Find the number of elements in the each array.

ii. Suppose Base(AAA) = 300 Base(BBB) = 500 & word per memory cell for AAA 2 words for memory a per memory cell for BBB & find the address of AAA[15], BBB[8], & BBB[0].

Sol: i) Number of elements

n = (upper bond - lower bond + 1)

for AAA(5:50)

n = 50 - 5 + 1 = 46

for BBB(-5:10)

n = 10 - (-5) + 1 = 16

AAA → 46 elements

BBB → 16 elements

ii) Address calculation

formula for address of an element in a 1D array

Address(A[i]) = Base(A) + (i - lower bond) × (No. of words per element)

D D M M Y Y Y Y

Given
Base (AAA) = 3
lower bond = 5
word size = 4
Address (AAA[i]) = 300 + (i-5)
A[15] = 300 + (15-5) × 4 = 300 + 40 = 340
A[55] = 300 + (55-5) × 4 = 300 + 200 = 500

for BBB(-5:10)
Base (BBB) = 500
lower bond = -5
word size = 2
Address [BBB(i)] = 500 + (i-(-5)) × 2 = 500 + (i+5) × 2
BBB[5] = 500 + (5+5) × 2 = 500 + 20 = 520
BBB[10] = 500 + (10+5) × 2 = 500 + 30 = 530

Array	Range	No of elements	element
AAA(5:50)	5 to 50	46	AAA[15]
			AAA[55]
BBB(-5:10)	-5 to 10	16	BBB[5]
			BBB[10]

10.

9. Convert the following infix expression into post fix expression using stacks.
Expression : A+(B*C - (D/B + F) * G) * H

Symbol	stack (operators)	postfix expression
+	+	A
C	+C	AC
B	+C	ACB

.	+C*	AB
C	+C*	AB
.	+C	ABC
-	+C-	ABC*
C	+C-C	ABC*
D	+C-C	ABC*
/	+C-C/	ABC*D
B	+C-C/	ABC*D
	+C-C+	ABC*DB
+	+C-C+	ABC*DB'*
F	+C-	ABC*DB'
*	+C-*	ABC*DB/F+G
G	+C-*	ABC*DB/F+G*-
	+C	ABC*DB/F+G*-
)		ABC*DB/F+G*-H
*	*	ABC*DB/F+G*-H*
H	*	ABC*DB/F+G*-H*
		+

10. Write all algorithm to evaluate a postfix expression & trace the algorithm for the following expression showing the stack contents.
Expression → 651 - /23 * + 11 + /

A:- Algorithm:-
* Initialize an empty stack.
* Scan the postfix expression from left to right.
* If the symbol is an operand, push it on to the stack
* If the symbol is an operator
* pop the top two elements from the stack.

DD MM YYYY

ii. Apply the operator : result = operand2 operator operand1

iii. push the result back on to the stack.

* After the entire expression is scanned, the final value in the stack is the result.

11. Illustrate the same for the given postfix expression
expression : ABC - D * EF / +
Given A = 6, B = 3, C = 2, D = 5, E = 1, F = 7

A:- Step by step Evaluation

Symbol	Stack	Operation
A = 6	6	push
B = 3	6, 3	push
C = 2	6, 3, 2	push
-	6, 1	3 - 2 = 1
D = 5	6, 1, 5	push
*	6, 5	1 x 5 = 5
+	11	6 + 5 = 11
E = 1	11, 1	push
F = 7	11, 1, 7	push
/	11, 0.142	1 ÷ 7 = 0.142
+	11.142	11 + 0.142 = 11.142

12. Develop a c program to implement insertion, deletion & display operations on linear queue.

A:- #include <stdio.h>
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;
void insert() {

```
    int item;
    if (rear == SIZE-1)
    printf ("Enter the element to insert:");
    scanf ("%d", &item);
    if (front == -1) front=0;
    rear++;
    queue[rear]= item;
    }
}
void display() {
if (front == -1 || front > rear)
printf ("Queue is empty\n");
else {
printf(" Queue elements are: ");
for(int i = front ; i <= rear ; i++)
printf ("%d", queue[i]);
printf ("\n");
}
}
int main () {
int choice;
do {
printf ("\n 1. Insert \n 2. Delete \n 3. Display \n 4. Ex
\n Enter your choice:");
scanf ("%d", &choice);
switch (choice) {
case 1: insert(); break;
case 2: delete(); break;
case 3: display(); break;
case 4: printf ("Exiting... \n"); break;
```

D D M M Y Y Y Y

```
default : printf("Invalid choice\n");
}
} while (choice != 4);
return 0;
}
```

13. write the C functions to add two polynomials using a circular singly linked list.

$P_1 = 5n^3 + 4n^2 + 7n + 3$

$P_2 = 6n^2 + 5$

A:-

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
int coeff;
int pow
struct Node * next;
};
struct Node * createNode (int coeff, int pow) {
struct Node * newNode = (struct Node *) malloc (size
    o of struct node));
new node -> coeff = coeff;
new Node -> pow = pow;
new Node -> next = NULL;
return new Node;
}
void insert (struct Node ** head, int coeff, int pow){
struct Node * newNode =
create Node (coeff, pow);
if (*head == NULL){
*head = newNode;
```

```
new Node -> next = *head;
} else {
Struct Node *temp = *head
while (temp -> next != *head)
temp -> next = new Node;
new Node -> next = *head;
}
}
Struct Node * add poly (Struct node * p1, Struct Node* p2){
Struct Node * res = NULL;
Struct Node *a=p1, *b=p2;
do {
if (a->pow == b->pow){
insert (& res, a -> coff + b -> coeff, a->pow);
a = a -> next; b=b -> next;
} else if (a->pow > b->pow){
insert (& res, a -> coff);
insert (& P1, 5, 3);
insert (& P1, 4, 2);
insert (& P1, 7, 1);
insert (& P1 3, 0);
insert (& P2 6, 2);
insert (& P2 5, 6);
printf ("P1 = "); display (P1);
printf ("P2 = "); display (P2);
Sum = add poly (P1, P2);
printf ("Sum = "); display (Sum);
return 0;
```

14. Develop a recursive C program for factorial of N

numbers.

A:
```
#include <stdio.h>
int factorial(int n);
if (n==0)
return 1;
else
return n * factorial(n-1);
}
int main() {
int n;
printf("Enter a number: ");
scanf("%d", &n);
printf("Factorial of %d is %d \n", n, factorial(n));
return 0;
}
```

15. Develop functions to implement insertion deletion and display operations of circular queue.

A:
```
#include <stdlib.h>
#define SIZE 5
int queue[SIZE]
int front = -1, rear = -1;
void insert() {
int item;
if (front == 0 && rear = SIZE-1)
if (rear == (front -1) % (SIZE -1))
printf("Queue over flow \n");
else {
printf("Enter element to insert: ");
```

D D M M Y Y Y Y

```
scanf ("%d", &item);
if (front == -1) front = rear = 0;
else if (rear == SIZE-1 && front != 0) rear = 0;
else rear++;
queue[rear] = item;
  }
}
void delete () {
if (front == -1)
printf ("Queue under flow \n");
else {
printf ("Deleted element : %d \n", queue[front]);
if (front == rear)
front = rear = -1;
else if (front == SIZE-1)
front = 0;
else
front++
  }
}
void display () {
if (front == -1)
printf ("Queue elements are : ");
if (rear >= front) {
for (int i = front; i <= rear; i++)
printf ("%d", queue[i]);
  }
  else {
for (int i = front; i < SIZE; i++)
printf ("%d", queue[i]);
```

```
for (int i=0; i<=rear; i++)
printf("%d", queue[i]);
}
printf("\n");
}
}
int main()
{
int choice;
do {
printf("\n1. Insert \n2. Delete \n3. Display \n4. Exit\n"
Enter your choice);
scanf("%d", choice);
case 1: Insert(); break;
case 2: delete(); break;
case 3: display(); break;
case 4: printf("Exiting....\n"); break;
default: printf("Invalid choice\n");
}
} while (choice != 4);
return 0;
}
```

16. Explain linked list eg Algorithm to demonstrate single & doubly linked list.

A:- A linked list is a collections of nodes connected by the pointers.
Each node contain data & a pointer to the next and or previous node

Types :-

Single linked list - one direction connection.

D D M M Y Y Y Y

Doubly linked list - nodes linked in both directions

Algorithm for SLL :-

1. Start
2. Create a node with data & pointer field
3. If list empty. Set head = new node.
4. Else transverse to end & link new node.
5. Stop.

Algorithm for DLL :-

1. Start
2. Create a node with data, prev, next.
3. If list is empty, head = new node.
4. Else transverse to last node & link :
   last -> next = new node; new node -> prev = last;
5. Stop.

17. Explain Queue, circular Queue & write a function for circular queue.

A :- Queue :- A linear data structure that follows FIFO operations :- enqueue, dequeue, display.

circular Queue :- last node connects back to the first making efficient use of memory.

function :- use the insertion, deletion & display function.